

32-BIT SINGLE CYCLE RISC-V PROCESSOR

Complete Technical Reference — RV32I ISA | Verilog HDL

Project Type	32-bit Single Cycle CPU — Digital Design Final Year Project
Architecture	RISC-V RV32I Base Integer Instruction Set Architecture
Implementation	Verilog HDL (IEEE 1364-2001 Standard)
Modules Covered	11 Modules: PC, IMem, RegFile, ImmGen, CU, ALU-CU, ALU, DMem, MUX×3, Adders, TB
Instructions Supported	R-type, I-type, Load, Store, Branch (B-type) — 20+ instructions
Simulation Tool	Xilinx Vivado / Icarus Verilog + GTKWave
Report Date	June 28, 2026

■ Abstract

This report presents a comprehensive technical analysis of a 32-bit Single Cycle RISC-V processor implemented in Verilog HDL, conforming to the RV32I Base Integer Instruction Set Architecture. The processor executes every instruction — from fetch to write-back — within a single clock cycle, making it an ideal reference design for understanding foundational computer architecture principles.

The design comprises eleven distinct hardware modules: a Program Counter (PC), a PC+4 incrementer, Instruction Memory (ROM), a 32×32-bit Register File, an Immediate Generator covering six instruction formats, a Main Control Unit, an ALU Control Unit, a 32-bit Arithmetic Logic Unit (ALU), a Data Memory module, three 2-to-1 multiplexers, a Branch Address Adder, a general-purpose adder, and a testbench. Together these components realise the classic five-stage conceptual pipeline — Fetch, Decode, Execute, Memory, Write-Back — collapsed into a single combinational path.

The report exhaustively documents each module's purpose, port descriptions, internal logic, hardware synthesis implications, supported RISC-V instructions, numerical worked examples, and engineering improvement opportunities. Subsequent sections provide a holistic view of the datapath and control path, timing analysis, design strengths, limitations, and advanced enhancement roadmap. This document is suitable for final-year project defence (viva), placement technical interviews, and IEEE-style portfolio submission.

Table of Contents

■ Abstract	2
■ 1. Processor Overview	7
1.1 Five Conceptual Stages	7
1.2 Module Hierarchy	7
■ 2. Module-by-Module Analysis	9
2.1.1 Purpose	9
2.1.2 Inputs and Outputs	9
2.1.3 Internal Logic Explanation	9
2.1.4 Hardware Interpretation	9
2.1.5 Design Reasoning	10
2.1.6 Supported Instructions	10
2.1.7 Example Walkthrough	10
2.1.8 Notes / Possible Improvements	10
2.2.1 Purpose	11
2.2.2 Inputs and Outputs	11
2.2.3 Internal Logic Explanation	11
2.2.4 Hardware Interpretation	11
2.2.5 Design Reasoning	11
2.2.6 Supported Instructions	11
2.2.7 Example Walkthrough	11
2.2.8 Notes / Possible Improvements	11
2.3.1 Purpose	13
2.3.2 Inputs and Outputs	13
2.3.3 Internal Logic Explanation	13
2.3.4 Instruction Set Loaded	13
2.3.5 Hardware Interpretation	14
2.3.6 Design Reasoning	14
2.3.7 Notes / Possible Improvements	14
2.4.1 Purpose	15
2.4.2 Inputs and Outputs	15
2.4.3 Internal Logic Explanation	15
2.4.4 Hardware Interpretation	16
2.4.5 Design Reasoning	16
2.4.6 Supported Instructions	16
2.4.7 Example Walkthrough	16
2.4.8 Notes / Possible Improvements	16
2.5.1 Purpose	17

2.5.2 Inputs and Outputs	17
2.5.3 Internal Logic — Format-by-Format	17
2.5.4 Immediate Format Table	17
2.5.5 Hardware Interpretation	18
2.5.6 Design Reasoning	18
2.5.7 Example Walkthrough	18
2.5.8 Notes / Possible Improvements	18
2.6.1 Purpose	20
2.6.2 Inputs and Outputs	20
2.6.3 Control Signal Truth Table	20
2.6.4 Internal Logic Explanation	20
2.6.5 Hardware Interpretation	21
2.6.6 Design Reasoning	21
2.6.7 Supported Instructions	21
2.6.8 Notes / Possible Improvements	21
2.7.1 Purpose	22
2.7.2 Inputs and Outputs	22
2.7.3 ALU Control Decode Table	22
2.7.4 Internal Logic Explanation	22
2.7.5 Hardware Interpretation	23
2.7.6 Design Reasoning	23
2.7.7 Example Walkthrough	23
2.7.8 Notes / Possible Improvements	23
2.8.1 Purpose	24
2.8.2 Inputs and Outputs	24
2.8.3 Internal Logic Explanation	24
2.8.4 Hardware Interpretation	25
2.8.5 Design Reasoning	25
2.8.6 Supported Instructions	25
2.8.7 Example Walkthrough	25
2.8.8 Notes / Possible Improvements	25
2.9.1 Purpose	26
2.9.2 Inputs and Outputs	26
2.9.3 Internal Logic Explanation	26
2.9.4 Hardware Interpretation	27
2.9.5 Supported Instructions	27
2.9.6 Example Walkthrough	27
2.9.7 Notes / Possible Improvements	27
2.10.1 Purpose	28

2.10.2 Inputs and Outputs (all three share the same structure)	28
2.10.3 Internal Logic	28
2.10.4 Hardware Interpretation	28
2.10.5 Design Reasoning	28
2.10.6 Example Walkthrough — mux_1	29
2.10.7 Notes / Possible Improvements	29
2.11.1 Purpose	30
2.11.2 Inputs and Outputs	30
2.11.3 Internal Logic	30
2.11.4 Hardware Interpretation	30
2.11.5 Design Reasoning	30
2.11.6 Example Walkthrough	30
2.11.7 Notes / Possible Improvements	31
2.12.1 Purpose	32
2.13.1 Purpose	33
2.13.2 Key Testbench Elements	33
2.13.3 Monitored Internal Signals	33
2.13.4 Notes / Possible Improvements	33
■ 3. Datapath Flow	35
3.1 R-Type Instruction Flow (e.g., add x13, x16, x25)	35
3.2 Load Instruction Flow (e.g., lw x8, 15(x2))	35
3.3 Store Instruction Flow (e.g., sw x14, 12(x6))	35
3.4 Branch Instruction Flow (e.g., beq x9, x9, offset)	35
■ 4. Control Path Flow	37
4.1 Control Signal Generation Pipeline	37
4.2 PCSrc Logic	37
■ 5. Instruction Set Supported	38
5.1 R-Type Instructions	38
5.2 I-Type Instructions (Immediate Arithmetic)	38
5.3 Load Instructions	38
5.4 Store Instructions	39
5.5 Branch Instructions (B-Type)	39
5.6 U-Type & J-Type (Defined but Commented Out)	39
■ 6. Timing Analysis	40
6.1 Critical Path	40
6.2 Critical Path for Load Instruction (Longest Path)	40
6.3 Path Lengths by Instruction Type	40
6.4 Single-Cycle Timing Implication	40
■ 7. Design Strengths	41

■ 8. Design Limitations	42
■ 9. Suggested Improvements	43
9.1 Immediate Bug Fixes (Priority)	43
9.2 Feature Additions	43
9.3 Pipelining (Advanced — Major Enhancement)	43
■ 10. Expected Viva Questions & Technical Answers	44
■ 11. Register File Initial Values (Simulation State)	46
■ 12. Conclusion	47
■ References	48

1. Processor Overview

The 32-bit Single Cycle RISC-V processor is a hardware implementation of the RV32I ISA in which every instruction is fetched, decoded, executed, and written back within a single clock cycle. This architecture is the foundational building block taught in computer organisation courses and forms the basis upon which pipelined, superscalar, and out-of-order processors are built.

1.1 Five Conceptual Stages

Stage	Description	Key Modules Involved
1. Instruction Fetch (IF)	PC drives instruction memory. The 32-bit instruction at address PC is read combinationally. PC is updated to PC+4 (or branch target) on every rising clock edge.	program_counter, pcplus4, instruction_mem, mux_3
2. Instruction Decode (ID)	The 32-bit instruction word is decomposed. The opcode selects control signals. rs1/rs2 fields index the register file; the immediate generator produces a sign-extended immediate.	reg_file, Immediate_generator, controlunit
3. Execute (EX)	The ALU performs the arithmetic/logic operation. One input is a register value; the other is either a second register or the sign-extended immediate (selected by ALUSrc MUX). The Branch Adder computes PC+offset for conditional branches.	alu_unit, alu_control, mux_1, Branch_Adder
4. Memory Access (MEM)	For load instructions, data memory is read at the ALU-computed address. For store instructions, rs2 data is written to that address. All other instructions pass through this stage unchanged.	data_memory
5. Write-Back (WB)	The result — either the ALU output or data loaded from memory — is written back to the destination register rd, controlled by the MemtoReg MUX and RegWrite signal.	mux_2, reg_file

1.2 Module Hierarchy

The top-level module (Top_Module, not shown in the source but instantiated in the testbench) wires all sub-modules together. The hierarchy is:

```
Top_Module
  program_counter (PC register)
  pcplus4 (PC + 4 adder)
  instruction_mem (32x64 ROM)
  reg_file (32x32 register bank)
  Immidiate_generator (sign-extend)
  controlunit (main control)
  alu_control (ALU select)
  alu_unit (ALU core)
  data_memory (32x64 RAM)
  mux_1 (ALUSrc select)
  mux_2 (MemtoReg select)
  mux_3 (PC next select)
```

```
    Branch_Adder (PC + offset)
    adder_logic (general adder)
```


2. Module-by-Module Analysis

2.1 Module: program_counter

2.1.1 Purpose

The Program Counter (PC) is the heart of instruction sequencing. It stores the 32-bit memory address of the *currently executing* instruction. Every clock cycle, the PC is updated to the next instruction address — either PC+4 for sequential flow or a branch/jump target for control transfer. Without the PC, the processor has no way of knowing which instruction to fetch next.

2.1.2 Inputs and Outputs

Signal	Width	Type	Description
clk	1 bit	Input	System clock. PC updates on the rising edge (posedge).
rst	1 bit	Input	Synchronous active-high reset. Forces pc_out to 0x00000000.
pc_in	32 bit	Input	Next PC value. Comes from mux_3 (either PC+4 or branch target).
pc_out	32 bit	Output (reg)	Current PC value. Feeds instruction memory read address.

2.1.3 Internal Logic Explanation

```
always @(posedge clk or posedge rst) begin
  if (rst)
    pc_out <= 32'b00; // Reset: go to address 0
  else
    pc_out <= pc_in; // Normal: capture next PC
end
```

always @(posedge clk or posedge rst) — This is a synchronous sequential block that is sensitive to the *rising edge* of the clock and the *rising edge* of reset. The dual sensitivity makes reset *synchronous* to the clock domain (it only takes effect on the next posedge clk when rst=1).

if (rst) pc_out <= 32'b00 — When reset is asserted, the PC is forced to address 0. This ensures that after power-on or a system reset, the processor always begins fetching from the first instruction in memory (I_Mem[0]).

else pc_out <= pc_in — On every other rising clock edge (normal operation), the PC captures the value of pc_in. The non-blocking assignment (<=) ensures glitch-free sequential behaviour by scheduling the update at the end of the time step.

2.1.4 Hardware Interpretation

- Synthesises to a **32-bit D flip-flop register** (32 DFFs in parallel).
- The reset path adds a **multiplexer before the D input**: Sel=rst drives 0; Sel=0 passes pc_in.
- The entire block is **synchronous sequential** — updates only on clock edges.
- Estimated gate cost: ~32 DFFs + 32-bit 2-to-1 MUX for reset = ~160 equivalent gates.

2.1.5 Design Reasoning

The PC must be a register (not combinational) because the processor needs to *remember* its current position across clock cycles. A combinational PC would cause it to change the moment any input changes, creating feedback oscillations. The PC is 32 bits wide to address up to 4 GB of memory (2^{32} bytes), consistent with the RV32I 32-bit address space.

2.1.6 Supported Instructions

The PC module is involved in **every single instruction** — it drives the fetch of each opcode.

2.1.7 Example Walkthrough

Scenario: `rst=1` at `t=0`. At `t=12 ns` `rst` falls to 0. `pc_in` = 0x00000004 (from PC+4).

`t=0`: `rst=1` → `pc_out` = 0x00000000

`t=10 ns` (posedge): `rst=1` → `pc_out` stays 0x00000000

`t=20 ns` (posedge): `rst=0`, `pc_in=0x4` → `pc_out` = 0x00000004

`t=30 ns` (posedge): `rst=0`, `pc_in=0x8` → `pc_out` = 0x00000008

2.1.8 Notes / Possible Improvements

WARNING: The reset is coded as asynchronous in sensitivity list (posedge `rst`) but behaves synchronously because `rst` is checked only inside the posedge `clk` block. This is a common but potentially confusing style. For true asynchronous reset, the `if(rst)` branch would need to be guarded differently. Best practice: decide consciously and document clearly.

- Improvement: Add a parameterised `RESET_VECTOR` to start from any address.
- Improvement: Use *parameter* for width (e.g., `DATA_WIDTH=32`) for portability.

2.2 Module: pcplus4

2.2.1 Purpose

For sequential (non-branch, non-jump) instruction execution, the PC must advance to the next instruction, which is always 4 bytes ahead in the RV32I ISA (all instructions are 32-bit fixed-width = 4 bytes). This module computes PC+4 purely combinationally, providing the default next-PC value.

2.2.2 Inputs and Outputs

Signal	Width	Type	Description
fromPc	32 bit	Input	Current PC value (pc_out from program_counter).
NexttoPc	32 bit	Output	PC + 4. Fed to mux_3 as the sequential next PC.

2.2.3 Internal Logic Explanation

```
assign NexttoPc = 4 + fromPc;
```

A single **continuous assignment** (assign) computes the sum of the constant 4 and the current PC. This is purely combinational: any change in fromPc instantly propagates to NexttoPc with only propagation delay.

2.2.4 Hardware Interpretation

- Synthesises to a **32-bit ripple-carry adder** (or carry-lookahead adder depending on tool).
- One operand is the constant 4 (binary: ...0100), which allows the synthesis tool to **optimise away** the upper bits of the adder — only bits [2:0] actually perform addition.
- Fully **combinational** — no registers, no clock.

2.2.5 Design Reasoning

RV32I specifies that all base instructions are 32 bits (4 bytes) wide. Therefore, sequential execution always moves to address PC+4. The constant increment is separated into its own module for clarity and reuse. In a pipelined processor, this adder would sit in the Fetch stage.

2.2.6 Supported Instructions

Provides the default next PC for **all instructions** except taken branches and jumps (JAL/JALR).

2.2.7 Example Walkthrough

```
fromPc = 0x00000008 (Instruction 2, I_Mem[2])  
NexttoPc = 0x00000008 + 4 = 0x0000000C → feeds mux_3 input A
```

2.2.8 Notes / Possible Improvements

- Consider merging with adder_logic (which is functionally identical) to avoid redundancy.

- For RVC (Compressed Instructions) support, this would need to conditionally add 2 or 4.

2.3 Module: instruction_mem

2.3.1 Purpose

The Instruction Memory models a **read-only memory (ROM)** that stores the program (machine code). The processor fetches the 32-bit instruction at the address provided by the PC. In a real SoC, this would be Flash or on-chip ROM; in simulation, it is initialised via an *initial* block.

2.3.2 Inputs and Outputs

Signal	Width	Type	Description
clk	1 bit	Input	Clock (connected but unused for reads — combinational ROM).
rst	1 bit	Input	Reset (connected but unused — ROM content is fixed).
read_addr	32 bit	Input	Byte address from PC. Divided by 4 to get word index.
instruction_out	32 bit	Output (reg)	32-bit instruction word fetched from I_Mem[PC>>2].

2.3.3 Internal Logic Explanation

```
reg [31:0] I_Mem[0:63]; // 64-word (256 byte) ROM array

// Combinational read
always @(*) begin
    instruction_out = I_Mem[read_addr >> 2]; // word-aligned: byte addr ÷ 4
end

// ROM initialisation
initial begin
    for (i = 0; i < 64; i = i + 1) I_Mem[i] = 32'h00000000;
    I_Mem[1] = 32'b00000000_11001_10000_000_01101_0110011; // add x13,x16,x25
    I_Mem[2] = 32'b0100000_00011_01000_000_00101_0110011; // sub x5,x8,x3
    ... (14 instructions total)
end
```

reg [31:0] I_Mem[0:63] — Declares a 64-element array of 32-bit registers. This is Verilog's way of modelling a memory array. It will synthesise to a **ROM** when only written in an initial block (simulation-only).

always @(*) — Combinational sensitivity list. read_addr changes → instruction_out updates immediately. No clock needed for read operations: this models an asynchronous ROM.

read_addr >> 2 — Right-shift by 2 = integer divide by 4. Converts the byte address from the PC into a word index. PC=0→I_Mem[0], PC=4→I_Mem[1], PC=8→I_Mem[2], etc.

initial block — Runs once at simulation time 0. Clears all 64 words to NOP (32'h00000000), then loads the 15 test instructions. This block is *not synthesisable* for FPGAs; it would be replaced with a .mif or .coe file.

2.3.4 Instruction Set Loaded

Index	Binary (Hex)	Assembly	Type
I_Mem[0]	0x00000000	NOP	—
I_Mem[1]	0x01980_6B3	add x13, x16, x25	R-type
I_Mem[2]	0x403402B3	sub x5, x8, x3	R-type
I_Mem[3]	0x0031_70B3	and x1, x2, x3	R-type
I_Mem[4]	0x0051_E233	or x4, x3, x5	R-type
I_Mem[5]	0x0051_C233	xor x4, x3, x5	R-type
I_Mem[6]	0x0051_70_93	andi x1, x2, 5	I-type
I_Mem[7]	0x003_464_93	ori x9, x8, 3	I-type
I_Mem[8]	0x004_464_93	xori x9, x8, 4	I-type
I_Mem[9]	0x005_504_83	lb x9, 5(x10)	Load
I_Mem[10]	0x003_191_83	lh x9, 3(x3)	Load
I_Mem[11]	0x00F_12_403	lw x8, 15(x2)	Load
I_Mem[12]	0x00F_18_423	sb x15, 8(x3)	Store
I_Mem[13]	0x00E_31_523	sh x14, 10(x6)	Store
I_Mem[14]	0x00E_32_623	sw x14, 12(x6)	Store

2.3.5 Hardware Interpretation

- Synthesises to a **synchronous/asynchronous ROM** or **block RAM (BRAM)** on FPGA.
- Combinational read path: **asynchronous read** behaviour (output follows address with propagation delay only).
- No write port: true ROM behaviour during normal operation.

2.3.6 Design Reasoning

The instruction memory is kept separate from data memory (Harvard architecture). This allows simultaneous instruction fetch and data memory access in the same cycle — critical for a single-cycle processor that must complete all stages in one clock period. In a von Neumann architecture both would share one bus, serialising these accesses.

2.3.7 Notes / Possible Improvements

WARNING: The initial block is simulation-only. For FPGA synthesis, use `$readmemh("program.hex", I_Mem)` or Vivado's COE/MIF file initialisation to load instructions into BRAM.

WARNING: Only 64 instructions supported. For real programs, this is extremely limiting. Increase to at least 1024 entries or use external memory.

- The I_Mem[0] entry has 31 bits instead of 32 in the source — minor typo, synthesises as NOP anyway.
- Add `$readmemb/$readmemh` support for flexible program loading without recompilation.

2.4 Module: reg_file

2.4.1 Purpose

The Register File is the processor's scratchpad — a bank of 32 general-purpose 32-bit registers (x0–x31) that hold operands and results. Every arithmetic, logical, load, and store instruction reads from or writes to this file. It provides two simultaneous read ports and one write port, enabling all three operands (rs1, rs2, rd) to be accessed in the same clock cycle.

2.4.2 Inputs and Outputs

Signal	Width	Type	Description
clk	1 bit	Input	Clock for synchronous write.
rst	1 bit	Input	Reset (not fully implemented in write logic — see notes).
Reg_write	1 bit	Input	Write enable. When 1 and Rd≠0, writes write_data to register Rd.
Rs1	5 bit	Input	Source register 1 address (0–31). Selects read_data1.
Rs2	5 bit	Input	Source register 2 address (0–31). Selects read_data2.
Rd	5 bit	Input	Destination register address. Target for write_data.
write_data	32 bit	Input	Data to write into register Rd (from WB MUX).
read_data1	32 bit	Output	Asynchronous read of register Rs1.
read_data2	32 bit	Output	Asynchronous read of register Rs2.

2.4.3 Internal Logic Explanation

```
reg [31:0] Registers[0:31]; // 32 × 32-bit register bank

// Synchronous write (with x0 hardwire)
always @(posedge clk or posedge rst) begin
  if (Reg_write && Rd != 0) begin
    Registers[Rd] <= write_data;
  end
end

// Asynchronous read with x0 always = 0
assign read_data1 = (Rs1 == 0) ? 32'b0 : Registers[Rs1];
assign read_data2 = (Rs2 == 0) ? 32'b0 : Registers[Rs2];
```

always @(posedge clk) — Write is synchronous. A write takes effect on the rising clock edge, meaning the new value is available in the *next* clock cycle. This prevents combinational loops: the same cycle's read and write cannot conflict.

if (Reg_write && Rd != 0) — Two-condition write gate. Reg_write must be asserted (enabled by control unit) AND the destination must not be x0. This is the RISC-V architectural requirement that register x0 is always zero and cannot be written.

assign read_data1 = (Rs1 == 0) ? 32'b0 : Registers[Rs1] — Asynchronous (combinational) read. If Rs1=0, output is hardwired to 0 regardless of memory content. This guarantees x0=0 even if the initial block sets Registers[0] to something else. Reads are instant (within propagation delay).

2.4.4 Hardware Interpretation

- Synthesises to a **32×32-bit register bank** (1024 flip-flops total).
- Write port: a 32-to-1 decoder selects Rd, enabling one register's DFFs with write_data on clock edge.
- Read ports: two 32-to-1 MUXes select Rs1 and Rs2 outputs combinationaly.
- x0 guard: a comparator (Rs1/Rs2 == 5'b0) drives a 2-to-1 MUX to clamp output to 0.
- Total: approximately 1024 DFFs + 2× 32:1 MUX + 1× 32:1 decoder + 2× comparators.

2.4.5 Design Reasoning

RISC-V mandates that x0 is the *zero register* — reads always return 0, writes are silently discarded. This simplifies many operations: mv rd, rs is encoded as addi rd, rs, 0, and discarding a result uses x0 as Rd. The separate read/write sensitivity (async read, sync write) matches the standard textbook register-file design.

2.4.6 Supported Instructions

All R-type, I-type, Load, and Store instructions read from this file. R-type, I-type, and Load instructions write results back.

2.4.7 Example Walkthrough

Instruction: add x13, x16, x25 (I_Mem[1])
 Rs1 = 16 → read_data1 = Registers[16] = 40
 Rs2 = 25 → read_data2 = Registers[25] = 65
 ALU computes: 40 + 65 = **105**
 Rd = 13, Reg_write=1 → on next posedge clk: Registers[13] = 105

2.4.8 Notes / Possible Improvements

WARNING: The rst signal is in the sensitivity list but the always block has no if(rst) branch inside the write logic — so rst has no effect on register writes. This means on reset, register values are NOT cleared. Add an explicit reset branch or remove rst from sensitivity.

- Improvement: support write-first or read-first semantics explicitly for same-cycle read/write to Rd.
- Improvement: parameterise REG_WIDTH and NUM_REGS for easy scaling.

2.5 Module: Immediate_generator (Immediate Generator)

2.5.1 Purpose

Most RISC-V instructions embed a constant (immediate) directly in the instruction word. However, different instruction formats scatter immediate bits across different bit positions, and the immediate must be **sign-extended** from its native width to 32 bits before use. The Immediate Generator extracts and reassembles these bits based on the instruction's opcode, then sign-extends to produce a 32-bit value ready for the ALU or address calculation.

2.5.2 Inputs and Outputs

Signal	Width	Type	Description
instruction	32 bit	Input	Full 32-bit instruction word.
ImmExt	32 bit	Output (reg)	32-bit sign-extended immediate value.

2.5.3 Internal Logic — Format-by-Format

```
always @(*) begin
  case(instruction[6:0]) // decode opcode

    // I-type: ANDI, ORI, XORI, ADDI, etc. (opcode 0010011)
    7'b0010011: ImmExt = {{20{instruction[31]}}, instruction[31:20]};

    // Load-type: LB, LH, LW (opcode 0000011)
    7'b0000011: ImmExt = {{20{instruction[31]}}, instruction[31:20]};

    // S-type: SB, SH, SW (opcode 0100011)
    7'b0100011: ImmExt = {{20{instruction[31]}}, instruction[31:25], instruction[11:7]};

    // B-type: BEQ, BNE (opcode 1100011)
    7'b1100011: ImmExt = {{19{instruction[31]}}, instruction[7], instruction[30:25],
      instruction[11:8], 1'b0};

    // U-type: LUI (opcode 0110111)
    7'b0110111: ImmExt = {instruction[31:12], 12'b0};

    // U-type: AUIPC (opcode 0010111)
    7'b0010111: ImmExt = {instruction[31:12], 12'b0};

    // J-type: JAL (opcode 1101111)
    7'b1101111: ImmExt = {{11{instruction[31]}}, instruction[19:12], instruction[20],
      instruction[30:21], 1'b0};

    default: ImmExt = 32'b0;
  endcase
end
```

2.5.4 Immediate Format Table

Format	Opcode	Bit Fields Used	Purpose	Bit[31] Sign-Extended?
I-type	0010011 / 0000011	inst[31:20]	ALU immediate / load offset	Yes — 20 copies of bit[31]
S-type	0100011	inst[31:25] ++ inst[11:7]	Store offset (split across fields)	Yes — 20 copies of bit[31]
B-type	1100011	inst[31], inst[7], inst[30:25], inst[11:8], 0	Branch offset (multiples of 2)	Yes — 19 copies of bit[31]
U-type	0110111 / 0010111	inst[31:12]	LUI / AUIPC upper 20 bits	No — lower 12 bits = 0
J-type	1101111	inst[31], inst[19:12], inst[20], inst[30:21], 0	JAL jump offset (multiples of 2)	Yes — 11 copies of bit[31]

2.5.5 Hardware Interpretation

- Purely **combinational** — a large multiplexer tree selecting and reassembling bits.
- The $\{n\{\text{bit}\}\}$ replication operator synthesises to **n-bit sign extension wires** (same wire fanned out n times — zero gates).
- The case statement synthesises to a **7-input decoder** driving a 7-to-1 MUX on the output.

2.5.6 Design Reasoning

RISC-V deliberately places the sign bit (MSB of the immediate) at instruction bit[31] across ALL formats. This means sign extension hardware is always the same — just replicate bit[31] — regardless of format. The bit scrambling of B-type and J-type ensures the branch/jump target computation aligns naturally with the adder without extra shifters.

2.5.7 Example Walkthrough

Instruction: `andi x1, x2, 5` → 32'b000000000101_00010_111_00001_0010011

opcode = 7'b0010011 (I-type)

inst[31:20] = 000000000101 = 12'd5

Sign bit inst[31] = 0 (positive)

ImmExt = {20{1'b0}, 000000000101} = 0x00000005 = **+5**

Instruction: `sw x14, 12(x6)` → 32'b00000000_01110_00110_010_01100_0100011

opcode = 7'b0100011 (S-type)

inst[31:25] = 00000000, inst[11:7] = 01100

ImmExt = {20{0}, 00000000, 01100} = 0x0000000C = **+12**

2.5.8 Notes / Possible Improvements

WARNING: The module source is missing its closing 'end' and 'endmodule' — the case block is not properly terminated. This is a syntax error that will prevent compilation. Always verify that every begin has a matching end.

- ADDI (add immediate) opcode is not included — only ANDI/ORI/XORI-family covered. Add 7'b0010011 handling already covers ADDI since it shares the opcode — but ensure the case is complete.
- Consider using a function instead of always/case for purely combinational logic — cleaner synthesis.

2.6 Module: controlunit (Main Control Unit)

2.6.1 Purpose

The Main Control Unit is the processor's decision-making centre. Given the 7-bit opcode field of the fetched instruction, it generates a set of control signals that govern the behaviour of every datapath element for that instruction: which multiplexer input to select, whether to read or write memory, whether to write a register, and what class of ALU operation to perform.

2.6.2 Inputs and Outputs

Signal	Width	Type	Description
instruction	7 bit	Input	Opcode field (bits [6:0]) of the 32-bit instruction.
ALUSrc	1 bit	Output (reg)	0=RegB goes to ALU, 1=Immediate goes to ALU.
MemtoReg	1 bit	Output (reg)	0=ALU result written to Rd, 1=Mem data written to Rd.
Reg_write	1 bit	Output (reg)	1=Enable register file write on this cycle.
Memread	1 bit	Output (reg)	1=Enable data memory read (load instructions).
MemWrite	1 bit	Output (reg)	1=Enable data memory write (store instructions).
branch	1 bit	Output (reg)	1=This is a branch instruction (AND with Zero for PC select).
ALUOp	2 bit	Output (reg)	00=ADD (loads/stores), 10=R-type/I-type, 11=Branch (SUB).

2.6.3 Control Signal Truth Table

Opcode	Instruction Type	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
0110011	R-type (ADD,SUB,AND...)	0	0	1	0	0	0	10
0000011	Load (LB,LH,LW)	1	1	1	1	0	0	00
0100011	Store (SB,SH,SW)	1	X	0	0	1	0	00
1100011	Branch (BEQ,BNE)	0	X	0	0	0	1	11
0010011	I-type (ANDI,ORI,XORI)	1	0	1	0	0	0	10
default	Unknown	0	0	0	0	0	0	00

2.6.4 Internal Logic Explanation

```

always@(*) begin
  case(instruction) // instruction = 7-bit opcode
    7'b0110011: {ALUSrc,MemtoReg,Reg_write,Memread,MemWrite,branch,ALUOp}
      <= 9'b001000_10; // R-type
    7'b0000011: {ALUSrc,MemtoReg,Reg_write,Memread,MemWrite,branch,ALUOp}
      <= 9'b111100_00; // Load
  endcase
end

```

```
// ... etc.  
endcase  
end
```

The module uses a **case statement on the 7-bit opcode** to select the correct signal bundle. All 9 control signals are assigned simultaneously via a concatenation. The use of **non-blocking assignment** (`<=`) inside a combinational block is technically a coding style issue (should be blocking `=`) but most simulators handle it correctly.

2.6.5 Hardware Interpretation

- Synthesises to a **combinational decoder**: a 7-to-9 bit lookup table.
- Equivalent to a **ROM-based control unit** (PLA or LUT on FPGA).
- No registers, no clock — purely combinational; output changes with opcode.

2.6.6 Design Reasoning

RISC-V was designed so that the opcode field alone is sufficient to determine the instruction format and most control signals. This allows the control unit to decode the instruction in parallel with the register file read — both happen in the decode stage simultaneously, reducing the critical path.

2.6.7 Supported Instructions

Controls datapath for: ADD, SUB, AND, OR, XOR (R-type); ANDI, ORI, XORI (I-type); LB, LH, LW (Load); SB, SH, SW (Store); BEQ, BNE (Branch).

2.6.8 Notes / Possible Improvements

WARNING: Non-blocking assignments (`<=`) in an `always@(*)` combinational block can cause simulation mismatches. Use blocking assignments (`=`) in combinational `always` blocks.

- U-type (LUI, AUIPC) and J-type (JAL) opcodes are not handled — add these for completeness.
- Add JALR (opcode 1100111) support for register-indirect jumps.

2.7 Module: alu_control (ALU Control Unit)

2.7.1 Purpose

While the main control unit distinguishes broad instruction types (R-type vs Load vs Store), the ALU Control Unit fine-tunes the exact ALU operation required. For R-type instructions, funct3 and funct7 differentiate ADD from SUB, AND from OR, SLL from SRL, etc. This two-level control hierarchy keeps the main control unit simple while giving full ALU flexibility.

2.7.2 Inputs and Outputs

Signal	Width	Type	Description
ALUOp	2 bit	Input	From main control: 00=ADD forced, 10=use funct, 11=SUB forced.
fun7	7 bit	Input	Instruction bits [31:25]. Differentiates ADD/SUB, SRL/SRA.
fun3	3 bit	Input	Instruction bits [14:12]. Specifies operation type.
Control_out	4 bit	Output (reg)	4-bit ALU operation select sent to alu_unit.

2.7.3 ALU Control Decode Table

ALUOp	funct7	funct3	Control_out	ALU Operation	Instruction
10	0000000	000	0000	ADD	ADD, ADDI
00	0000000	000/001/010	0000	ADD	LW/SW/LB/SB (address calc)
10	0100000	000	0001	SUB	SUB
10	0000000	111	0010	AND	AND, ANDI
10	0000000	110	0011	OR	OR, ORI
10	0000000	100	0100	XOR	XOR, XORI
10	0000000	001	0101	SLL	SLL
10	0000000	101	0110	SRL	SRL
10	0100000	101	0111	SRA	SRA
10	0000000	010	1000	SLT	SLT

2.7.4 Internal Logic Explanation

```
always@(*) begin
  case({ALUOp, fun7, fun3}) // 12-bit concatenation
    12'b10_0000000_000: Control_out <= 4'b0000; // ADD (R-type)
    12'b00_0000000_000: Control_out <= 4'b0000; // ADD (loads)
    12'b10_0100000_000: Control_out <= 4'b0001; // SUB
    12'b10_0000000_111: Control_out <= 4'b0010; // AND
    // ... etc.
```

```
endcase
end
```

The key insight is the **12-bit case key**: {ALUOp[1:0], fun7[6:0], fun3[2:0]}. This combines the instruction class (from main CU) with the fine-grained function code. When ALUOp=00 (loads/stores), any funct values produce ADD (for address calculation). When ALUOp=10, funct7/funct3 determine the specific R-type/I-type operation.

2.7.5 Hardware Interpretation

- Synthesises to a **12-to-4 bit priority encoder / LUT**.
- Purely **combinational** — no registers.
- The 12-bit key space (4096 entries) is sparsely populated — synthesis tools optimise to a minimal LUT.

2.7.6 Design Reasoning

funct7 bit 30 (instruction[30]) is the critical differentiator between ADD and SUB, and between SRL and SRA. RISC-V encodes these as complementary pairs sharing the same funct3, differentiated only by funct7[5]. The two-level ALU control is a textbook Patterson & Hennessy design used in commercial processors.

2.7.7 Example Walkthrough

Instruction: sub x5, x8, x3
 funct7 = 0100000, funct3 = 000, ALUOp (from CU) = 10
 case key = {10, 0100000, 000} = 12'b10_0100000_000
 → Control_out = 4'b0001 = **SUB**

2.7.8 Notes / Possible Improvements

WARNING: The ALUOp=11 case (Branch, for SUB to compute zero flag for BEQ) is not explicitly handled — the default gives ADD. For correct BEQ behaviour, add case 12'b11_xxxxxxx_000: Control_out = 4'b0001 (SUB).

- Consider expanding to support SLTU (unsigned comparison) for unsigned branch operations.

2.8 Module: alu_unit (Arithmetic Logic Unit)

2.8.1 Purpose

The ALU is the computational core of the processor. It takes two 32-bit operands and a 4-bit control code, then performs one of nine arithmetic or logical operations, producing a 32-bit result and a 1-bit Zero flag. The Zero flag is critical for branch decisions (BEQ/BNE). Every instruction type — R, I, Load, Store, Branch — passes through the ALU.

2.8.2 Inputs and Outputs

Signal	Width	Type	Description
A	32 bit	Input	First ALU operand (always from rs1 register).
B	32 bit	Input	Second operand: rs2 (R-type) or sign-extended imm (I/Load/Store).
control_in	4 bit	Input	ALU operation code from alu_control.
zero	1 bit	Output (reg)	1 if ALU_out == 0, else 0. Used for BEQ branch condition.
ALU_out	32 bit	Output (reg)	Result of the ALU operation.

2.8.3 Internal Logic Explanation

```
always @(A or B or control_in) begin
  case (control_in)
    4'b0000: ALU_out = A + B; // ADD
    4'b0001: ALU_out = A - B; // SUB
    4'b0010: ALU_out = A & B; // AND
    4'b0011: ALU_out = A | B; // OR
    4'b0100: ALU_out = A ^ B; // XOR
    4'b0101: ALU_out = A << B[4:0]; // SLL
    4'b0110: ALU_out = A >> B[4:0]; // SRL
    4'b0111: ALU_out = $signed(A) >>> B[4:0]; // SRA
    4'b1000: ALU_out = ($signed(A) < $signed(B)) ? 1 : 0; // SLT
    default: ALU_out = 32'b0;
  endcase
  zero = (ALU_out == 32'b0) ? 1 : 0;
end
```

- 4'b0000 ADD:** Standard 32-bit two's-complement addition. Used for R-type ADD, ADDI, load/store address calculation.
- 4'b0001 SUB:** Two's-complement subtraction. For BEQ: if A-B=0 then Zero=1, indicating equality.
- 4'b0101 SLL:** Logical left shift by B[4:0] bits (only low 5 bits used — max shift 31, matching RV32I spec).
- 4'b0111 SRA:** \$signed(A) forces Verilog to treat A as signed, so >>> performs arithmetic right shift (MSB is replicated, not zeroed). This preserves the sign for negative numbers.
- 4'b1000 SLT:** Set Less Than — outputs 1 if A < B (signed comparison), else 0. Result is a boolean stored in a 32-bit register.

zero = (ALU_out == 32'b0): Computed *after* the case block, reflecting the final ALU_out. This single bit is AND-ed with the branch control signal to determine PC mux selection.

2.8.4 Hardware Interpretation

- **32-bit adder/subtractor** (shared hardware with SUB = ADD of two's complement).
- **32-bit barrel shifter** for SLL, SRL, SRA.
- **32-bit bitwise AND, OR, XOR gates.**
- **32-bit signed comparator** for SLT.
- **32-bit NOR tree** (or zero-detect) for the Zero flag.
- All outputs selected by a **9-to-1 MUX** (driven by control_in).
- Fully **combinational** — no registers, no clock.

2.8.5 Design Reasoning

The Zero flag from SUB is the most efficient way to check equality in hardware: $A==B \leftrightarrow A-B==0$. RISC-V BEQ uses this directly. The 5-bit shift amount (B[4:0]) limits shifts to 0–31, which is the full range possible for 32-bit values — any larger shift is architecturally meaningless.

2.8.6 Supported Instructions

ADD, ADDI (load/store addr), SUB, AND, ANDI, OR, ORI, XOR, XORI, SLL, SRL, SRA, SLT. Also computes BEQ/BNE condition via SUB+Zero flag.

2.8.7 Example Walkthrough

Instruction: sub x5, x8, x3
 Registers: x8=2, x3=12
 A=2, B=12, control_in=4'b0001 (SUB)
 ALU_out = 2 - 12 = -10 (0xFFFFFFFF6 in two's complement)
 zero = (0xFFFFFFFF6 == 0) = **0** (not zero)

Instruction: beq x9, x9, offset (BEQ checking $x9==x9$)
 A=Registers[9]=1, B=Registers[9]=1, control_in=0001 (SUB)
 ALU_out = 1-1 = 0, zero = **1** → branch taken

2.8.8 Notes / Possible Improvements

WARNING: The always block sensitivity list uses explicit signal names (A or B or control_in) rather than always@(*). This is error-prone — if you add a signal and forget to update the list, simulation will be incorrect. Always use always@(*) for combinational blocks.

WARNING: The 'zero' assignment is outside the case block but still inside the always block. This is correct but could be moved to a separate continuous assign for clarity.

- Add SLTU (unsigned SLT) and XOR-based equality check for BEQ optimisation.
- Implement overflow/carry flag outputs for future exception handling.

2.9 Module: data_memory (Data Memory)

2.9.1 Purpose

Data Memory is the read-write storage used by load and store instructions. Load instructions (LB, LH, LW) read data from this memory into a register; store instructions (SB, SH, SW) write register data into this memory. The ALU computes the effective address (base + offset) which is used as the memory index. This memory models the processor's data RAM.

2.9.2 Inputs and Outputs

Signal	Width	Type	Description
clk	1 bit	Input	Clock for synchronous write operations.
rst	1 bit	Input	Active-high reset: clears all memory to 0.
Mem_Write	1 bit	Input	Write enable (from control unit MemWrite signal).
Mem_Read	1 bit	Input	Read enable (from control unit Memread signal).
Read_addr	32 bit	Input	Effective memory address from ALU output.
Write_Data	32 bit	Input	Data from rs2 register to write into memory.
Mem_data_out	32 bit	Output	Data read from memory (to MemtoReg MUX).

2.9.3 Internal Logic Explanation

```
reg [31:0] D_Memory [63:0]; // 64 word × 32-bit RAM

// Combinational read (gated by Mem_Read)
assign Mem_data_out = (Mem_Read) ? D_Memory[Read_addr] : 32'b00;

// Hardcoded initialisation on every clock edge (BUG – see notes)
always @(posedge clk) begin
    D_Memory[17] = 56;
    D_Memory[15] = 65;
    D_Memory[28] = 88;
end

// Synchronous write with reset
always @(posedge clk or posedge rst) begin
    if (rst) begin
        for (i=0; i<64; i=i+1) D_Memory[i] = 32'b00;
    end else if (Mem_Write) begin
        D_Memory[Read_addr] = Write_Data;
    end
end
```

assign Mem_data_out = (Mem_Read) ? D_Memory[Read_addr] : 32'b00 — Combinational, gated read. If Mem_Read=0 (non-load instruction), output is forced to 0 to prevent undefined propagation downstream.

First always block (init): This is a significant design bug — it hardcodes three memory locations to fixed values on every single clock edge. This means any store instruction that writes to addresses 15, 17, or 28

will have its data overwritten on the very next clock cycle. This block should be replaced with an initial block.

Second always block (write): Correct synchronous write pattern. On reset, all 64 words are cleared. On `Mem_Write=1`, `Write_Data` is stored at `Read_addr`. Note: blocking assignment (`=`) is used inside sequential logic — should be non-blocking (`<=`) to avoid synthesis/simulation discrepancies.

2.9.4 Hardware Interpretation

- Synthesises to a **single-port synchronous RAM** ($64 \times 32 = 2048$ bits).
- On FPGA: maps to a **Block RAM (BRAM)** or distributed LUT-RAM.
- Read path: **asynchronous** (combinational assign).
- Write path: **synchronous** (registered on posedge clk).

2.9.5 Supported Instructions

Reads: `LB x9, 5(x10)` → `address = x10+5 = 23+5 = 28` → reads `D_Memory[28] = 88`.

Reads: `LH x9, 3(x3)` → `address = x3+3 = 12+3 = 15` → reads `D_Memory[15] = 65`.

Reads: `LW x8, 15(x2)` → `address = x2+15 = 2+15 = 17` → reads `D_Memory[17] = 56`.

Writes: `SW/SH/SB` — write `rs2` data to effective address.

2.9.6 Example Walkthrough

Instruction: `lw x8, 15(x2)` → `base=x2=2`, `offset=15`
 ALU computes: `2+15 = 17` → `Read_addr = 17`
`Mem_Read=1` (Load control signal)
`Mem_data_out = D_Memory[17] = 56`
 → 56 is written back to `x8` via `MemtoReg MUX`

2.9.7 Notes / Possible Improvements

WARNING: CRITICAL BUG: The first `always @(posedge clk)` block reinitialises `D_Memory[15]=65`, `D_Memory[17]=56`, `D_Memory[28]=88` on every clock edge. This will override any stores to these addresses. Replace with 'initial begin ... end' block.

WARNING: The memory uses word-address indexing (`D_Memory[Read_addr]`) without byte-address alignment. The ALU output is a byte address (same as instruction memory's approach requires `>>2` shift). Add alignment logic for correct byte/halfword/word addressing.

WARNING: `LB/LH/SB/SH` require byte/halfword extraction — only the lower bytes should be read/written. Currently, the module always reads/writes full 32-bit words regardless of instruction width.

- Fix: Use blocking→non-blocking for synchronous writes.
- Fix: Add byte-enable signals for `SB/SH` support.

2.10 Modules: mux_1, mux_2, mux_3 (2-to-1 Multiplexers)

2.10.1 Purpose

Multiplexers (MUXes) are the routing elements of the datapath. They select between two possible data sources based on a 1-bit control signal. The three MUXes in this design handle three critical datapath decisions every cycle.

MUX	Sel Signal	Input A (Sel=0)	Input B (Sel=1)	Datapath Role
mux_1	ALUSrc	read_data2 (rs2 register)	ImmExt (immediate)	Second ALU operand: register vs immediate
mux_2	MemtoReg	ALU_out (computation result)	Mem_data_out (loaded data)	Write-back: ALU result vs loaded memory data
mux_3	PCSrc (branch AND zero)	PC+4 (sequential)	Branch target (PC+offset)	Next PC: sequential vs branch taken

2.10.2 Inputs and Outputs (all three share the same structure)

Signal	Width	Type	Description
SelN	1 bit	Input	Select: 0→output=AN, 1→output=BN.
AN	32 bit	Input	Input port A (selected when Sel=0).
BN	32 bit	Input	Input port B (selected when Sel=1).
MuxN_out	32 bit	Output	Selected output (continuously driven).

2.10.3 Internal Logic

```
assign MuxN_out = (SelN == 1'b0) ? AN : BN; // Ternary operator → 2:1 MUX
```

A single **continuous assignment** using the ternary operator. This is the most concise and synthesis-friendly way to express a 2-to-1 multiplexer. The output is combinational — it changes within a propagation delay whenever any input changes.

2.10.4 Hardware Interpretation

- Each MUX synthesises to a **32-bit 2-to-1 multiplexer** (32 2:1 MUX gates).
- Fully **combinational** — no registers, no clock.
- On FPGA: maps to LUT resources (each LUT can implement a 2:1 MUX).

2.10.5 Design Reasoning

mux_1 (ALUSrc): R-type instructions use rs2 as the second ALU input; I-type, Load, and Store instructions use the sign-extended immediate. This single MUX allows the same ALU to serve both instruction classes.

mux_2 (MemtoReg): For loads, the result written to rd is data from memory; for all other instructions, it is the ALU result. A MUX cleanly selects between the two sources.

mux_3 (PCSrc): When a branch is not taken (or for non-branch instructions), PC advances to PC+4. When a branch is taken (branch=1 AND zero=1 for BEQ), the PC jumps to the branch target computed by Branch_Adder.

2.10.6 Example Walkthrough — mux_1

Instruction: andi x1, x2, 5 (I-type)

ALUSrc = 1 (immediate) → mux_1 selects B1 = ImmExt = 5

ALU computes: Registers[2] AND 5 = 2 AND 5 = 0 (binary: 00...010 AND 00...101 = 0)

Instruction: and x1, x2, x3 (R-type)

ALUSrc = 0 (register) → mux_1 selects A1 = read_data2 = Registers[3] = 12

2.10.7 Notes / Possible Improvements

- All three MUXes are identical — consider a parameterised mux module with WIDTH parameter to reduce code duplication.
- For JAL/JALR support, mux_2 would need a third input (PC+4 for the return address). This requires upgrading to a 3-to-1 MUX or adding a fourth MUX.

2.11 Module: Branch_Adder

2.11.1 Purpose

The Branch Adder computes the **branch target address** for conditional branches (BEQ, BNE). In RISC-V, branch targets are PC-relative: the target = PC + sign-extended-offset. The immediate generator produces the offset (already shifted left by 1 in B-type encoding), and this adder computes the sum to determine where execution should jump if the branch is taken.

2.11.2 Inputs and Outputs

Signal	Width	Type	Description
PC	32 bit	Input	Current program counter value (pc_out).
offset	32 bit	Input	Sign-extended branch offset from Immediate Generator.
branch_target	32 bit	Output (reg)	Computed branch target: PC + offset.

2.11.3 Internal Logic

```
always @(*) begin
  branch_target <= PC + (offset); // PC-relative branch target
end
```

A combinational always@(*) block computing PC + offset. The result is the absolute address of the branch target in memory. Note: non-blocking assignment in a combinational block (style issue — see notes). The offset is already the full sign-extended immediate from the Immediate Generator, which has embedded the LSB=0 requirement for 2-byte alignment.

2.11.4 Hardware Interpretation

- Synthesises to a **32-bit adder**.
- Purely **combinational**.
- output = reg but behaves as wire due to always@(*) — synthesises without flip-flops.

2.11.5 Design Reasoning

PC-relative branch addressing means the branch offset in the instruction encodes the *distance* from the current instruction, not an absolute address. This makes the code **position-independent** — if the program is loaded at a different base address, all branch targets automatically adjust. The B-type immediate's LSB is hardwired to 0, ensuring all branch targets are 2-byte aligned (even for 16-bit Compressed instructions).

2.11.6 Example Walkthrough

Instruction: BEQ x9, x9, 12 (B-type)
PC = 0x00000060 (address of this instruction)
B-type immediate = +12 (encoded in instruction bits)
branch_target = 0x60 + 12 = 0x6C
If x9==x9 (zero=1, branch=1) → PC_next = 0x6C (branch taken)

2.11.7 Notes / Possible Improvements

WARNING: *Non-blocking assignment (<=) inside always@(*) is a style error. Use blocking (=).*

- This module is functionally identical to adder_logic — consider consolidating to a single generic adder module with two instances.
- For JAL/JALR, an additional adder (or reuse this one) computes the jump target.

2.12 Module: adder_logic (General Purpose Adder)

2.12.1 Purpose

A simple general-purpose 32-bit adder. In this design, it appears to serve as a utility module for computing sums where needed in the datapath — for example, as an alternate or supplementary path for address computation or PC updates. It is the simplest module in the design.

Signal	Width	Type	Description
in_1	32 bit	Input	First addend.
in_2	32 bit	Input	Second addend.
Sum_out	32 bit	Output	Sum: in_1 + in_2.

```
assign Sum_out = in_1 + in_2;
```

Identical combinational 32-bit addition. Synthesises to a 32-bit ripple or carry-lookahead adder.

***NOTE:** This module is identical to pcplus4 (which adds constant 4) and Branch_Adder (same logic). In a clean design, one parameterised adder module would be instantiated multiple times.*

2.13 Module: Top_Module_tb (Testbench)

2.13.1 Purpose

The testbench is not synthesisable hardware — it is a simulation wrapper that applies stimuli (clock, reset) to the DUT (Device Under Test: Top_Module) and monitors the internal signals to verify correct processor behaviour. It is the verification environment for the design.

2.13.2 Key Testbench Elements

```
always #5 clk = ~clk; // 10 ns clock period = 100 MHz

initial begin
  $dumpfile("cpu.vcd"); // VCD waveform output
  $dumpvars(0, Top_Module_tb); // Dump all signals

  $monitor(...); // Continuous terminal logging

  clk = 0; rst = 1;
  #12 rst = 0; // Reset for 12 ns (1 full + partial cycle)

  #3000; // Run 300 cycles
  $finish;
end
```

always #5 clk = ~clk — Generates a 10 ns period clock (100 MHz). The clock toggles every 5 ns (half period).

\$dumpfile / \$dumpvars — Generates a VCD (Value Change Dump) file that can be opened in GTKWave to view signal waveforms for debugging.

\$monitor — Prints a line to the console whenever any monitored signal changes, showing time, PC, instruction, ALU output, memory output, and writeback data — the five most critical signals to verify correct operation.

rst = 1 for 12 ns, then = 0 — Holds reset for slightly more than one full clock cycle (posedge at 10 ns sees rst=1, posedge at 20 ns sees rst=0), ensuring the PC is cleanly initialised before the first real instruction executes.

2.13.3 Monitored Internal Signals

Signal	Meaning
uut.PC_top	Current PC — confirms sequential fetch
uut.instruction_top	Fetches instruction word
uut.ALU_out_top	ALU result — computation verification
uut.Mem_data_out_top	Data memory output — load verification
uut.write_back_top	Value written to register — WB verification

2.13.4 Notes / Possible Improvements

WARNING: *The Top_Module itself (instantiated as uut) is not present in the provided source code — only the testbench references it. The top-level wiring module must be created separately.*

- Add \$display assertions to automatically verify expected register values.
- Add individual test cases with explicit expected-vs-actual checking for each instruction.
- Consider using SystemVerilog assertions (SVA) for formal verification coverage.

■ 3. Datapath Flow

The datapath is the collection of hardware elements through which data flows during instruction execution. In a single-cycle processor, all these flows happen within one clock period. Below is the step-by-step data flow for each instruction type.

3.1 R-Type Instruction Flow (e.g., add x13, x16, x25)

```
1. FETCH: PC → instruction_mem → instruction [32-bit opcode at PC]
2. DECODE: instruction[19:15]=Rs1=16, [24:20]=Rs2=25, [11:7]=Rd=13
   reg_file reads Reg[16]=40, Reg[25]=65
   Immediate_generator: opcode=R-type → ImmExt not used
   Control: ALUSrc=0, MemtoReg=0, RegWrite=1, MemRead=0, MemWrite=0
3. EXECUTE: mux_1: ALUSrc=0 → B_ALU = read_data2 = 65
   alu_control: {10, 00000000, 000} → Control_out = 0000 (ADD)
   alu_unit: 40 + 65 = 105, zero=0
4. MEMORY: MemRead=0, MemWrite=0 → data_memory bypassed
   Mem_data_out = 0 (not used)
5. WRITEBACK: mux_2: MemtoReg=0 → write_back = ALU_out = 105
   reg_file: RegWrite=1, Rd=13 → Reg[13] = 105
6. PC UPDATE: mux_3: branch=0 → PC_next = PC+4
```

3.2 Load Instruction Flow (e.g., lw x8, 15(x2))

```
1. FETCH: PC → instruction_mem[PC>>2] → instruction
2. DECODE: Rs1=2 (base), Rd=8, imm=15
   reg_file reads Reg[2]=2
   Control: ALUSrc=1, MemtoReg=1, RegWrite=1, MemRead=1
3. EXECUTE: mux_1: ALUSrc=1 → B_ALU = ImmExt = 15
   ALU: 2 + 15 = 17 (effective address)
4. MEMORY: MemRead=1 → Mem_data_out = D_Memory[17] = 56
5. WRITEBACK: mux_2: MemtoReg=1 → write_back = 56
   reg_file: Rd=8 → Reg[8] = 56
6. PC UPDATE: PC_next = PC+4
```

3.3 Store Instruction Flow (e.g., sw x14, 12(x6))

```
1. FETCH: instruction fetched as normal
2. DECODE: Rs1=6 (base), Rs2=14 (data), imm=12
   Control: ALUSrc=1, RegWrite=0, MemRead=0, MemWrite=1
3. EXECUTE: mux_1: ALUSrc=1 → B_ALU = 12 (S-type imm)
   ALU: Reg[6]=44 + 12 = 56 (write address)
4. MEMORY: MemWrite=1 → D_Memory[56] = Reg[14] = 20
5. WRITEBACK: RegWrite=0 → no register written
6. PC UPDATE: PC_next = PC+4
```

3.4 Branch Instruction Flow (e.g., beq x9, x9, offset)

```
1. FETCH: instruction fetched
2. DECODE: Rs1=9, Rs2=9, imm=branch_offset
   Control: ALUSrc=0, RegWrite=0, branch=1, ALUOp=11
   Branch_Adder: PC + ImmExt (B-type) = branch_target
3. EXECUTE: ALU: Reg[9] - Reg[9] = 0, zero=1
```

4. PCSrc: $\text{branch AND zero} = 1 \text{ AND } 1 = 1 \rightarrow \text{branch taken}$
5. PC UPDATE: $\text{mux_3: Sel}=1 \rightarrow \text{PC_next} = \text{branch_target}$

■ 4. Control Path Flow

The control path generates the signals that route data through the datapath. It is entirely combinational in a single-cycle processor, meaning control signals are generated within the same clock cycle as the instruction is fetched.

4.1 Control Signal Generation Pipeline

```

instruction[6:0] (opcode)
↓
controlunit
↓
ALUSrc  ████████████████████████████████████████ → mux_1 (selects ALU operand B)
MemtoReg ████████████████████████████████████████ → mux_2 (selects WB data)
RegWrite ████████████████████████████████████████ → reg_file (enable write)
MemRead  ████████████████████████████████████████ → data_memory (enable read)
MemWrite  ████████████████████████████████████████ → data_memory (enable write)
branch   ████████████████████████████████████████ → AND gate with zero flag
ALUOp[1:0] ████████████████████████████████████████ → alu_control
↓
instruction[31:25] (funct7)
instruction[14:12] (funct3)
↓
Control_out[3:0]
↓
alu_unit
↓
zero flag
↓
branch AND zero → PCSrc → mux_3

```

4.2 PCSrc Logic

The PCSrc signal that drives mux_3 is not a dedicated module — it is the AND of two signals:

```
PCSrc = branch AND zero
```

This implements BEQ semantics: take the branch IF (it IS a branch instruction) AND (the ALU result IS zero, meaning $rs1 == rs2$). For BNE, the logic would be: branch AND (NOT zero). The current design only supports BEQ-style branching.

■ 5. Instruction Set Supported

5.1 R-Type Instructions

Instruction	Syntax	Operation	funct7	funct3	Opcode
ADD	add rd, rs1, rs2	$rd = rs1 + rs2$	0000000	000	0110011
SUB	sub rd, rs1, rs2	$rd = rs1 - rs2$	0100000	000	0110011
AND	and rd, rs1, rs2	$rd = rs1 \text{ AND } rs2$	0000000	111	0110011
OR	or rd, rs1, rs2	$rd = rs1 \text{ OR } rs2$	0000000	110	0110011
XOR	xor rd, rs1, rs2	$rd = rs1 \text{ XOR } rs2$	0000000	100	0110011
SLL	sll rd, rs1, rs2	$rd = rs1 \ll rs2[4:0]$	0000000	001	0110011
SRL	srl rd, rs1, rs2	$rd = rs1 \gg rs2[4:0]$	0000000	101	0110011
SRA	sra rd, rs1, rs2	$rd = rs1 \ggg rs2[4:0]$ (arith)	0100000	101	0110011
SLT	slt rd, rs1, rs2	$rd = (rs1 < rs2) ? 1 : 0$	0000000	010	0110011

5.2 I-Type Instructions (Immediate Arithmetic)

Instruction	Syntax	Operation	funct3	Opcode
ADDI	addi rd, rs1, imm	$rd = rs1 + \text{sign_ext}(imm)$	000	0010011
ANDI	andi rd, rs1, imm	$rd = rs1 \text{ AND } \text{sign_ext}(imm)$	111	0010011
ORI	ori rd, rs1, imm	$rd = rs1 \text{ OR } \text{sign_ext}(imm)$	110	0010011
XORI	xori rd, rs1, imm	$rd = rs1 \text{ XOR } \text{sign_ext}(imm)$	100	0010011
SLLI	slli rd, rs1, shamt	$rd = rs1 \ll \text{shamt}$	001	0010011
SRLI	srli rd, rs1, shamt	$rd = rs1 \gg \text{shamt}$	101	0010011
SRAI	srai rd, rs1, shamt	$rd = rs1 \ggg \text{shamt}$ (arith)	101	0010011
SLTI	slti rd, rs1, imm	$rd = (rs1 < imm) ? 1 : 0$	010	0010011

5.3 Load Instructions

Instruction	Syntax	Operation	funct3	Width Loaded
LB	lb rd, imm(rs1)	$rd = \text{sign_ext}(\text{Mem}[rs1+imm][7:0])$	000	8-bit byte
LH	lh rd, imm(rs1)	$rd = \text{sign_ext}(\text{Mem}[rs1+imm][15:0])$	001	16-bit halfword
LW	lw rd, imm(rs1)	$rd = \text{Mem}[rs1+imm][31:0]$	010	32-bit word

5.4 Store Instructions

Instruction	Syntax	Operation	funct3	Width Stored
SB	sb rs2, imm(rs1)	$\text{Mem}[\text{rs1}+\text{imm}][7:0] = \text{rs2}[7:0]$	000	8-bit byte
SH	sh rs2, imm(rs1)	$\text{Mem}[\text{rs1}+\text{imm}][15:0] = \text{rs2}[15:0]$	001	16-bit halfword
SW	sw rs2, imm(rs1)	$\text{Mem}[\text{rs1}+\text{imm}] = \text{rs2}[31:0]$	010	32-bit word

5.5 Branch Instructions (B-Type)

Instruction	Syntax	Operation	funct3	Condition
BEQ	beq rs1, rs2, offset	if $(\text{rs1} == \text{rs2})$ PC += offset	000	ALU zero=1 after SUB
BNE	bne rs1, rs2, offset	if $(\text{rs1} != \text{rs2})$ PC += offset	001	ALU zero=0 after SUB

5.6 U-Type & J-Type (Defined but Commented Out)

Instruction	Syntax	Operation	Type	Status
LUI	lui rd, imm	$\text{rd} = \text{imm} \ll 12$	U-type	Imm gen defined, CU not handled
AUIPC	auipc rd, imm	$\text{rd} = \text{PC} + (\text{imm} \ll 12)$	U-type	Imm gen defined, CU not handled
JAL	jal rd, offset	$\text{rd} = \text{PC} + 4; \text{PC} += \text{offset}$	J-type	Imm gen defined, CU/datapath incomplete

■ 6. Timing Analysis

6.1 Critical Path

In a single-cycle processor, the critical path determines the minimum clock period — it is the longest combinational path from any register output to any register input. The clock period must be long enough for the slowest instruction to complete.

6.2 Critical Path for Load Instruction (Longest Path)

```
PC Register (output)
→ Instruction Memory (async read: ~2 ns)
→ Control Unit (combinational decode: ~0.5 ns)
→ Register File (async read rs1: ~1.5 ns)
→ mux_1 (ALUSrc select: ~0.3 ns)
→ ALU (32-bit ADD for addr: ~3 ns)
→ Data Memory (async read: ~2 ns)
→ mux_2 (MemtoReg select: ~0.3 ns)
→ Register File Write Port setup time
```

Total critical path: ~9.6 ns → max freq ≈ 100 MHz (rough estimate)

6.3 Path Lengths by Instruction Type

Instruction Type	Key Path Stages	Estimated Delay	Bottleneck
R-type (ADD)	PC→IMem→RegFile→ALU→RegFile	~7 ns	ALU computation
I-type (ANDI)	PC→IMem→ImmGen→ALU→RegFile	~7.5 ns	ALU + immediate routing
Load (LW)	PC→IMem→RegFile→ALU→DMem→RegFile	~9.6 ns	Data memory read
Store (SW)	PC→IMem→RegFile→ALU→DMem	~8 ns	Data memory write
Branch (BEQ)	PC→IMem→RegFile→ALU→PCSrc→PC	~7 ns	ALU zero + MUX

6.4 Single-Cycle Timing Implication

Because all instructions must complete in *one* clock cycle, the cycle time must accommodate the **slowest instruction** — the LW (Load Word), which has the longest path. This means faster instructions (R-type, Branch) waste time waiting for the clock edge. In a 5-stage pipeline, each stage takes only 1/5th of this time, enabling 5× higher throughput.

■ 7. Design Strengths

- **Educational clarity:** Each hardware concept (PC, Register File, ALU, Control) is a separate module, making the architecture easy to understand and trace.
- **Harvard architecture:** Separate instruction and data memories allow simultaneous access, correctly modelling the single-cycle fetch + memory access requirement.
- **Correct x0 handling:** The register file hardwires x0 to zero via both the combinational read (ternary operator) and write guard ($Rd \neq 0$), consistent with the RV32I specification.
- **Complete immediate formats:** The Immediate Generator correctly handles all six RISC-V immediate formats (I, S, B, U, U, J) with proper sign extension and bit reassembly.
- **Two-level ALU control:** The hierarchical $ALUSrc \rightarrow ALUOp \rightarrow ALUControl \rightarrow ALU$ design cleanly separates instruction-class decisions from fine-grained operation selection.
- **Comprehensive testbench:** The testbench generates VCD output for waveform analysis, uses \$monitor for continuous logging, and covers reset + 300 clock cycles of execution.
- **Rich test program:** 15 instructions covering all supported instruction types are pre-loaded, enabling immediate simulation without external file loading.
- **Combinational reads, synchronous writes:** The register file and instruction memory use the standard async-read / sync-write pattern, correct for single-cycle operation.

8. Design Limitations

#	Limitation	Impact	Affected Module
1	No pipelining — all 5 stages in one cycle	Low throughput; clock limited by LW path (~100 MHz vs 500+ MHz pipelined)	Entire design
2	Memory size: 64 instructions, 64 data words	Cannot run real programs; only toy test sequences	instruction_mem, data_memory
3	No hazard detection or forwarding	N/A in single-cycle, but critical limitation for pipeline upgrade	N/A (single-cycle)
4	No branch prediction	Every branch resolves in same cycle but kills throughput in pipeline	program_counter, mux_3
5	BEQ only for branches; BNE not fully wired	Cannot conditionally branch on inequality	controlunit, mux_3
6	JAL/JALR not implemented	Cannot call subroutines or implement function returns	controlunit, mux_3
7	LUI/AUIPC not connected	Cannot load large constants or build PC-relative addresses	controlunit
8	Data memory byte/halfword not handled (LB/LH/SB/SH)	LB/LH reads full 32 bits; SB/SH writes full 32 bits — incorrect behavior	data_memory
9	Data memory initialisation bug	D_Memory[15,17,28] overwritten every clock cycle	data_memory
10	Non-blocking in combinational blocks	Potential simulation/synthesis mismatch	controlunit, Branch_Adder, alu_control
11	Missing endmodule/end in some modules	Synthesis tool will throw parse errors	Immediate_generator, alu_unit, mux_3, data_memory
12	No exception/interrupt handling	Cannot respond to illegal instructions, traps, or external IRQs	Entire design
13	Simulation-only memory initialisation (initial blocks)	Not synthesisable for FPGA without COE/MIF files	instruction_mem, data_memory
14	Module name typo: 'Immediate_generator'	Minor; does not affect function but reduces professionalism	Immediate_generator

■ 9. Suggested Improvements

9.1 Immediate Bug Fixes (Priority)

- Fix missing endmodule/end in Immdiate_generator, alu_unit, mux_3, data_memory.
- Replace first always@(posedge clk) block in data_memory with initial begin...end.
- Change non-blocking (<=) to blocking (=) in all combinational always blocks.
- Add BNE support: PCSrc = branch AND (NOT zero) for BNE, AND zero for BEQ — requires funct3 decode.
- Fix data memory addressing: add Read_addr >> 2 word-alignment (or use byte enables).
- Add ALU control case for ALUOp=11 (branch SUB) explicitly.

9.2 Feature Additions

9.2.1 Complete RV32I ISA Support

- Add JAL: rd=PC+4, PC=PC+J-imm. Requires 3rd MUX input for PC+4 writeback to rd.
- Add JALR: rd=PC+4, PC=(rs1+I-imm)&~1. Requires extra PC source.
- Add LUI, AUIPC: connect to control unit, route through datapath.
- Add all branch types: BLT, BGE, BLTU, BGEU using signed/unsigned comparators.

9.2.2 Memory Improvements

- Increase memory to 1024+ entries, use \$readmemh for FPGA-compatible initialisation.
- Implement byte-enable signals for correct LB/LH/SB/SH operation.
- Add separate data and instruction caches for FPGA deployment.

9.3 Pipelining (Advanced — Major Enhancement)

Converting this single-cycle design to a 5-stage pipeline would dramatically increase throughput. The stages would be separated by pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB). This requires adding:

Enhancement	Description	Difficulty
5-Stage Pipeline	Add 4 sets of pipeline registers between stages	Medium
Data Forwarding	Forward EX/MEM and MEM/WB results back to EX stage to resolve RAW hazards	Medium-High
Hazard Detection Unit	Detect load-use hazards, insert bubbles (NOPs) as needed	Medium
Branch Prediction	Predict not-taken; flush pipeline on misprediction	High
Stall Control	Freeze IF/ID stages and insert NOP in EX on load-use	Medium
Exception Handling	CSR registers, trap handler, mtvec/mepc/mcause support	High
Cache Controller	L1 I-cache and D-cache with hit/miss logic	Very High
Cache	Instruction + Data L1 cache, reducing memory access latency	Very High

■ 10. Expected Viva Questions & Technical Answers

Q1: Why is x0 always zero in RISC-V?

x0 (zero register) is architecturally hardwired to zero. Writes to x0 are silently discarded (enforced by 'if Rd != 0' guard in reg_file). Reads always return 0. This simplifies pseudo-instructions: 'mv rd, rs' = 'addi rd, rs, 0', 'nop' = 'addi x0, x0, 0'.

Q2: Why does the PC increment by 4?

All RV32I base instructions are 32 bits (4 bytes) wide. Since memory is byte-addressed, the next instruction is always 4 bytes ahead. PC+4 is computed by pcplus4 using a simple adder.

Q3: Why is sign extension needed for immediates?

Immediates in the instruction word are shorter than 32 bits (12 bits for I-type). To use them in 32-bit ALU operations, they must be extended to 32 bits. Sign extension replicates the MSB to preserve the two's complement sign — positive immediates extend with 0s, negative with 1s.

Q4: What is the critical path and why does it matter?

The critical path is the longest combinational delay chain — in this processor, it's the LW path (IMem→RegFile→ALU→DMem→Regfile). The clock period must exceed this delay; it determines maximum operating frequency. In this design, ~10ns → ~100 MHz max.

Q5: Why use separate instruction and data memories?

The Harvard architecture allows simultaneous fetch (instruction memory) and data access (data memory) in one cycle. A von Neumann architecture shares one bus, making simultaneous access impossible — the processor would need to stall.

Q6: What does the Zero flag do for branches?

For BEQ, the ALU subtracts rs1-rs2. If rs1==rs2, result=0 and zero=1. This zero flag is AND-ed with the 'branch' control signal. If both are 1 (branch instruction AND equal operands), PCSrc=1, and mux_3 selects the branch target instead of PC+4.

Q7: Why is there both a Control Unit AND an ALU Control Unit?

Two-level control: the main CU decodes the opcode to determine instruction class (R, I, Load, Store, Branch) and sets high-level signals. The ALU CU then uses ALUOp + funct3/funct7 to select the exact ALU operation. This hierarchy keeps the main CU simple while allowing fine-grained ALU control.

Q8: What is the difference between blocking and non-blocking assignments?

Blocking (=): executes immediately, like C assignment — used in combinational logic. Non-blocking (<=): schedules update at end of time step — used in sequential (clocked) logic to model flip-flop behaviour without race conditions. Mixing them incorrectly causes simulation/synthesis mismatches.

Q9: What limitations does this single-cycle design have vs a pipelined design?

1) Throughput: single-cycle processes 1 instruction per N ns (N=critical path); pipeline processes ~1 instruction per N/5 ns. 2) Efficiency: fast instructions (R-type) wait for the slow LW path. 3) Power: higher switching activity per cycle in single-cycle. 4) Complexity: pipeline needs hazard detection, forwarding, and branch prediction.

Q10: How would you add support for JAL?

JAL needs: (1) PC+4 written to rd (add 3rd MUX on write-back, selecting PC+4 when JAL), (2) PC updated to PC+J-imm (add 3rd input to mux_3 for J-type target), (3) Control unit case for opcode 1101111 setting appropriate signals, (4) J-type immediate already supported by Immediate Generator.

11. Register File Initial Values (Simulation State)

The register file is pre-initialised for simulation with these values:

Reg	ABI	Dec	Hex	Reg	ABI	Dec	Hex
x0	zero	0	0x0000 0000	x16	a6	40	0x0000 0028
x1	ra	3	0x0000 0003	x17	a7	50	0x0000 0032
x2	sp	2	0x0000 0002	x18	s2	60	0x0000 003C
x3	gp	12	0x0000 000C	x19	s3	70	0x0000 0046
x4	tp	20	0x0000 0014	x20	s4	80	0x0000 0050
x5	t0	3	0x0000 0003	x21	s5	80	0x0000 0050
x6	t1	44	0x0000 002C	x22	s6	90	0x0000 005A
x7	t2	4	0x0000 0004	x23	s7	70	0x0000 0046
x8	s0	2	0x0000 0002	x24	s8	60	0x0000 003C
x9	s1	1	0x0000 0001	x25	s9	65	0x0000 0041
x10	a0	23	0x0000 0017	x26	s10	4	0x0000 0004
x11	a1	4	0x0000 0004	x27	s11	32	0x0000 0020
x12	a2	90	0x0000 005A	x28	t3	12	0x0000 000C
x13	a3	10	0x0000 000A	x29	t4	34	0x0000 0022
x14	a4	20	0x0000 0014	x30	t5	5	0x0000 0005
x15	a5	30	0x0000 001E	x31	t6	10	0x0000 000A

■ 12. Conclusion

This report has provided a comprehensive technical analysis of a 32-bit Single Cycle RISC-V processor implemented in Verilog HDL. The design successfully realises the RV32I base integer instruction set across 13 sub-modules, covering all five conceptual pipeline stages — Fetch, Decode, Execute, Memory, and Write-Back — within a single clock cycle.

The implementation demonstrates correct application of foundational digital design principles: synchronous sequential logic for state-holding elements (PC, Register File, Data Memory writes), combinational logic for data transformation (ALU, Immediate Generator, Control Units, MUXes), and the Harvard architecture for simultaneous instruction and data memory access.

Key learning outcomes from this project include: (1) how the RISC-V instruction encoding maps directly to hardware structures, (2) how a hierarchical control unit (Main CU → ALU CU → ALU) cleanly separates concerns, (3) why architectural choices like x0=zero and PC-relative branches simplify hardware, and (4) how the single-cycle critical path fundamentally limits processor performance.

Several bugs and limitations were identified — most critically, the data memory initialisation bug and missing endmodule statements — along with a clear roadmap for extending this design towards a pipelined, cache-enabled, full-RV32I implementation suitable for FPGA deployment.

This processor implementation serves as an excellent foundation for understanding modern CPU design and is well-suited for B.Tech final year project presentation, technical interview preparation, and IEEE-style portfolio documentation.

■ References

- [1] Patterson, D. A. & Hennessy, J. L. (2020). *Computer Organization and Design RISC-V Edition* (2nd ed.). Morgan Kaufmann.
- [2] Waterman, A. & Asanovic, K. (2019). *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V Foundation. <https://riscv.org/specifications/>
- [3] Harris, S. L. & Harris, D. (2021). *Digital Design and Computer Architecture RISC-V Edition*. Morgan Kaufmann.
- [4] IEEE Std 1364-2001 — IEEE Standard for Verilog Hardware Description Language.
- [5] Xilinx Vivado Design Suite User Guide: Synthesis (UG901).
- [6] RISC-V Green Card — Quick reference for RV32I/RV64I instruction encodings.
<https://inst.eecs.berkeley.edu/~cs61c/fa17/img/riscvcard.pdf>